JAVA PROJECT . DESIGN GUIDE

APPARAT

Web Interface, Login and Sessions

Servlet / JSP, AuthFilter, availability page, shared layout

Nazrin Nizzam

24BBS0111

WHAT THIS DOCUMENT GIVES YOU

- What was promised in Review 1 and what exists now
- How a request travels, and the login flow step by step
- The ideas behind each file, explained simply
- A demo script, the tests, and honest limits
- Fourteen likely viva questions with model answers

Every number in this guide was measured on the project (tests run, requests sent, files counted).

1. Your slice at a glance

In Review 1 your slice was **Web Interface and Servlet / JSP**. This table compares each promise with what the project contains today.

Review 1 item	Status	Where it lives
JSP pages	**Done**	login, home, availability, error pages added; resources, booking form and my bookings already existed
Servlet request/response flow	**Done**	every page follows Browser > Filter > Servlet > Service > DAO > JSP
Session-aware interface	**Done**	LoginServlet stores a SessionUser; the nav bar shows the name and role; links to staff and admin pages appear only for roles that hold the permission
Form validation	**Done**	browser checks in js/app.js, then the servlet and service check again on the server
UI design, CSS and JavaScript	**Done**	one shared css/app.css, one js/app.js, a shared nav fragment
Web error presentation	**Done**	one error.jsp for 400, 403, 404, 405, 409, 500, registered in web.xml
Real login for the whole site	**Done**	the three servlets that used a fixed user now read the logged-in user
Registration, password change, remember-me	**Not built**	not in Review 1 either; do not claim them

2. How a request travels

- **AuthFilter** runs before every servlet. It lets a few public pages through, sends strangers to the login page, and blocks staff-only pages for other roles.
- **Servlet** reads the request, calls one service method, then either forwards to a JSP (to show a page) or redirects (after a successful POST).
- **Service** holds the rules. **DAO** holds the SQL. The JSP never talks to the database.
- JSP files live under `WEB-INF/views`, so a browser cannot open them directly. Only a servlet can forward to them (verified: a direct request returns 404).

3. The login flow, step by step

#	What happens	File
1	Visitor asks for any page and has no session. AuthFilter redirects to /login.	filter/AuthFilter
2	LoginServlet (GET) shows the login form. The browser checks the email shape first.	servlet/LoginServlet, js/app.js
3	Form is sent by POST. The servlet passes email and password to AuthService.	servlet/LoginServlet
4	AuthService looks the email up (UserDao) and checks the password against the stored salted hash.	service/AuthService, util/PasswordUtil
5	Wrong email or wrong password both give the same message, so nobody can learn which emails exist.	service/AuthService

#	What happens	File
6	On success the servlet **invalidates any old session and creates a new one**, then stores a SessionUser (id, name, role).	servlet/LoginServlet
7	Redirect to /home. Later requests carry the session cookie, and AuthFilter lets them through.	servlet/HomeServlet
8	Logout is a POST. The session is invalidated, and the old cookie stops working.	servlet/LogoutServlet

4. Files you own

File	Job	Lines
filter/AuthFilter.java	applies the access decision to every request; re-reads the account each time	87
filter/AccessRules.java	which page needs which permission (pure logic, fully tested)	80
filter/SessionSupport.java	stores and reads the logged-in user in the session	49
servlet/BaseServlet.java	parent of servlets that need the logged-in user	31
servlet/LoginServlet.java	login form and login check; creates the new session	94
servlet/LogoutServlet.java	POST-only logout	32
servlet/HomeServlet.java	landing page after login	158
servlet/AvailabilityServlet.java	availability page for one instrument and day	90
service/AuthService.java	login rules	74
service/AuthenticationException.java	login failure type	8
service/AvailabilityService.java	builds the 18-slot day grid	109
dao/UserDao.java	reads a user and password hash by email	134
dao/AvailabilityDao.java	reads booked slot times for one day	60
dto/SessionUser.java	what the session remembers (never the hash)	13
dto/UserCredentials.java	user row plus hash, used only during login	16
dto/SlotView.java	one cell of the availability grid	13
util/PasswordUtil.java	PBKDF2 hashing and checking	105
webapp/WEB-INF/views/login.jsp	login page	54
webapp/WEB-INF/views/home.jsp	home page	160
webapp/WEB-INF/views/availability.jsp	availability page	95
webapp/WEB-INF/views/error.jsp	friendly error page	51
webapp/WEB-INF/views/fragments/nav.jsp	navigation bar shared by pages	38
webapp/css/app.css	shared styles, scoped by class	308
webapp/js/app.js	browser-side validation and confirmations	302
webapp/WEB-INF/web.xml	session timeout, error pages, welcome file	46
webapp/index.jsp	sends / to /home	3
Total		**2210**

5. The ideas behind it, in plain words

Authentication and authorization

Authentication asks *who are you?* (the login). Authorization asks *what may you do?* (roles). Each page that needs a permission is listed in AccessRules, and Role.can(...) says which roles hold it. AuthFilter applies the decision; see the separate Roles and Permissions guide.

Session and cookie

After login the server remembers you in an **HttpSession**. The browser holds only a random id in the JSESSIONID cookie. The cookie is marked HttpOnly (JavaScript cannot read it), and the session ends after 30 idle minutes (web.xml).

Password hashing

A password is never stored. We store **PBKDF2WithHmacSHA256** with a random **salt** and 120,000 iterations. The same password gives a different stored value every time, and the check uses a constant-time comparison.

Session fixation

If the session id stayed the same across login, an attacker who knew it beforehand would be logged in too. So login throws the old session away and starts a new one. Measured: the id before and after login was different.

Same message for both failures

Telling people 'no such email' would let someone test which emails exist. For an unknown email we still run a hash check, so the time taken is similar too.

Forward and redirect

A **forward** hands the same request to a JSP inside the server (used to show pages). A **redirect** tells the browser to ask again (used after a successful POST, so pressing refresh cannot repeat the action).

Why logout is a POST

A link can be opened by accident or by another site. A POST needs a deliberate button press, and GET /logout returns 405 (measured).

Escaping output (XSS)

Every value shown on a page goes through `c:out`, which turns < and > into harmless text. A name like x is shown as text, not run.

Browser checks versus server checks

js/app.js gives quick feedback, but a browser can be bypassed, so the server checks everything again. The browser check is convenience; the server check is protection.

Filters and the interface idea

AuthFilter **implements** the Jakarta `Filter` interface. Tomcat calls it before servlets. This is one real use of an interface in the project.

6. The availability page

Chooses an instrument and a date (default: today), then shows 18 half-hour rows from 09:00 to 18:00. A row is **Booked** if a row for that start time exists in `booking_slots`, and **Maintenance** if it falls inside a maintenance window (blocked for bookings). Cancelling a booking deletes its slot rows, so cancelled times show as **Free** again.

The grid is built by `AvailabilityService.buildGrid`, which touches no database, so unit tests can check it directly. Opening hours and slot length come from the constants in BookingService, so there is one place to change them.

7. The booking page and its messages

Choosing the time. The old page used a free-typing date-time box, which accepted times the rules then rejected. The page now has a **date picker** and two **dropdown lists**, one for the start and one for the end, containing only valid half-hour times (09:00 to 17:30 for starts, 09:30 to 18:00 for ends). The lists are built by the servlet from the same opening hours and slot length the booking rules use, so there is one place to change them.

- Picking a start suggests an end 30 minutes later, and the end list only offers later times.
- Moving the start past the chosen end clears the end.
- For today, start times that have already passed are disabled. If none are left, the page says *No times are left today. Please choose another date.*
- A one-line summary appears, for example *Electroscope (PRP-711), Tuesday, 27 October 2026, 10:00 - 11:30 (1 h 30 min)*.
- **Check availability** opens the availability page for the same instrument and day.
- After a successful booking the user lands on **My bookings** with a one-time message, such as *Booking #6181 confirmed: Electroscope on 2026-11-25, 10:00 - 11:30.* (stored in the session and shown once, so a URL cannot fake it).
- If something fails, the form comes back with everything the user typed still filled in.

Where each message comes from. Rules live in the Service, so every entrance gets the same explanation. `BookingService.findRangeError` returns the reason a time range breaks a rule; `isValidRange` simply asks it, so the rules exist once. The servlet adds messages about the request itself.

What went wrong	Status	What the user reads
No instrument / date / start / end chosen	400	Please choose an instrument. / a date. / a start time. / an end time.
Instrument id is not a number	400	Please choose an instrument from the list.
Date or time is not a real date or time	400	Please choose the date and times from the lists.
Start is in the past	400	That time has already passed. Please choose a later start time.
End is not after the start	400	The end time must be after the start time.
Shorter than 30 minutes	400	A booking must be at least 30 minutes long.
Start and end on different days	400	A booking must start and end on the same day.
Not on the hour or half hour	400	Bookings must start and end on the hour or half hour, for example 10:00 or 10:30.
Outside opening hours	400	The lab is open from 09:00 to 18:00. Please choose a time within these hours.
Instrument does not exist	400	That instrument does not exist.
Instrument is INACTIVE	400	This instrument is not available for booking right now.

What went wrong	Status	What the user reads
Part of the time is already booked	409	That time is no longer free: someone else has booked part of it. Check the availability page and pick another time.
Time falls in a maintenance window	409	The instrument is under maintenance at that time. Check the availability page for free times.
Database failure	500	The friendly error page, with no technical details

The dropdowns make most of these impossible to cause from the page. The messages still matter, because a request can be sent without using the page, and the server must explain itself either way. The JavaScript shows the same wording before the form is sent.

8. Small changes made to existing files

These are the only edits to code written earlier. Each is a few lines, and none changes behaviour except that the logged-in user is now used.

File	Change
BookingServlet, MyBookingsServlet, CancelBookingServlet	the fixed `userId = 1` now comes from the session user; one import added to each
resources.jsp, booking-form.jsp, my-bookings.jsp	the shared stylesheet and nav bar added, their own 'body' width rule removed, and the content wrapped in the shared page column, so every page has the same layout (before this, each page had a different width)
schema.sql (since removed)	was extended for login and maintenance; the whole database is now built by `database/setup.sql`
BookingService	`findRangeError` added (the same rules, with a plain-words reason); the vague 'Invalid booking time range.' messages replaced; instrument messages reworded
BookingServlet, booking-form.jsp	reworked for the date and time dropdowns, specific messages, and the confirmation on My bookings

The database is created by one file, `database/setup.sql`, which also adds the demo accounts below. The README explains how to run it. Tests use a separate database, apparat2_test.

9. Demo script (six minutes)

Demo accounts are **sample data for development only**:

Email	Password	Role
student@apparat.test	Student@123	STUDENT
faculty@apparat.test	Faculty@123	FACULTY
tech@apparat.test	Tech@123	TECHNICIAN
admin@apparat.test	Admin@123	ADMIN

- Open the site without logging in: you land on the login page.
- Try a wrong password: the message is the generic one. Log in as the student.
- Open Availability, pick an instrument and a day: all rows are Free.
- Open Book: pick an instrument and a date, then a start time. The end appears by itself and a summary shows the booking in words.
- Book 10:00-11:00 on that day. You land on My bookings with a confirmation. Reload Availability: two rows are now Booked.
- Log out, log in as faculty, try the same slot: you get the slot-taken message.

- As the student, try to open /manage/resources: a friendly 'Not allowed' page.
- Log in as the technician: the Manage and Reports links now appear.

10. What was tested

Test class	Tests	What it proves
PasswordUtilTests	6	right password verifies, wrong one does not; same password gives different hashes; garbage stored values never verify
AuthServiceDbTests	6	success returns the session user; wrong password and unknown email give the same message; blank input and users with no password are refused (uses a real database)
AvailabilityServiceTests	7	18 slots from 09:00 to 17:30; booked slots are marked; the closing time has no slot of its own; maintenance slots are marked separately from booked ones; a window covers every slot from its start to its end; windows on other days are ignored
BookingMessageTests	10	every booking rule gives its own plain-words reason (missing, end not after start, too short, different days, off the half hour, outside hours); isValidRange agrees with the reasons; createBooking and generateSlots pass the reason on
BookingInstrumentMessageDbTests	2	unknown instrument and INACTIVE instrument each get their own message (real database)

The whole project suite is now **154 tests, 0 failures** (24 existed before, 130 were added since).

Checks made against the running site (Tomcat, real requests): unauthenticated visit redirects to login; wrong and unknown logins give 401 with the same text; the session id changes at login; a student gets 403 for /manage and /reports; a booking made while logged in is saved under the right user; a second user is refused (403) when cancelling someone else's booking; after logout the old cookie no longer works; GET /logout returns 405; an unknown page shows the custom 404 page; direct access to a JSP returns 404. **Layout check in a real browser:** on all eight pages opened from the dashboard (home, instruments, book, my bookings, availability, manage, maintenance, reports), at 1200 px and 700 px wide, the nav bar, the content column, the heading position and the Log out button had identical size and position, with no sideways scrolling. **Booking page, in the browser:** choosing a start suggested the end and limited the end list to later times; moving the start past the end cleared it; for today the passed times were disabled; each submit error showed the same wording as the server. **Booking page, over HTTP:** every message in the table above was triggered and read back, the success path showed its confirmation once, the form kept the typed values after an error, and a script typed into the date field was shown as harmless text.

11. Honest limits (say these before the panel asks)

- **No CSRF tokens.** A malicious page could try to make a logged-in user's browser send a POST. A token per form would fix this; it is planned, not built.
- **No limit on login attempts**, so repeated guessing is not slowed down.
- **No registration or password change.** Users come from SQL (the demo file).
- **Roles are fixed in code** (an enum). Adding a fifth role is a code change, not something an administrator can do on a page.
- **The 'time has passed' rule is in the servlet and the browser, not in the Service.** Putting it in the Service would break the existing tests, which use fixed dates. Another entrance that calls the Service directly would not get this check.
- Availability is read when the page loads; it does not refresh by itself, and two people can still pick the same free slot (the booking step then reports the conflict).

12. Viva questions with model answers

Q1. What happens when I type a URL without being logged in?

AuthFilter runs first. It finds no SessionUser in the session and redirects to /login. Only /login, /css and /js are open.

Q2. Why do you store a hash and not the password?

If the database leaks, the passwords must not be readable. A salted PBKDF2 hash cannot be turned back into the password, and the salt makes equal passwords look different.

Q3. What is a salt, and why is it random?

A salt is random data mixed into the hash. Because it is different for every user, an attacker cannot reuse one pre-computed table of hashes.

Q4. What is session fixation, and how do you prevent it?

An attacker plants a known session id and waits for you to log in. We invalidate the old session on login and create a new one, so the planted id becomes useless.

Q5. Why is the error message the same for a wrong email and a wrong password?

A different message would reveal which emails exist. We also verify against a dummy hash for unknown emails so response time does not give it away.

Q6. Forward or redirect after login, and why?

Redirect. After a successful POST a redirect makes the browser do a fresh GET, so refreshing the page does not resend the login form.

Q7. Where is authorization checked?

In AuthFilter for staff-only paths (/manage, /reports), and in the Service for data ownership (a user can only cancel their own booking).

Q8. What stops a script in an instrument name from running?

Every value is printed with c:out, which escapes HTML. The page shows the characters as text.

Q9. Why is logout a POST?

So that opening a link, or another site, cannot log someone out. A GET to /logout returns 405.

Q10. Which Java interface does your code implement?

AuthFilter implements jakarta.servlet.Filter. Tomcat calls its doFilter method before every request.

Q11. Does validation in JavaScript protect the system?

No. It only helps the user. The servlet and service validate again, because anyone can send a request without using our page.

Q12. How does the booking page stop invalid times?

It offers only valid half-hour times in dropdown lists, the end list only offers times after the start, and passed times are disabled for today. The servlet and the Service then check again, because a request can be sent without the page.

Q13. Why does each failure have its own message, and where do they come from?

'Invalid booking' does not tell a user what to change. findRangeError in the Service returns the specific reason, so every entrance gets the same explanation; the servlet adds messages about the request, such as a missing field.

Q14. What would you add next?

CSRF tokens on every POST form, a limit on login attempts, and password change. Each is a known gap, listed above.